

DAY-2

Student Name: Saloni Kumari

Branch: MCA

Subject Name: LG 3

UID: 25MCA20087

Section/Group: MCA-3

Date of Performance: 08-07-2026

Q1. Implement Strassen's Matrix Multiplication algorithm to compute the product of two square matrices.

Code:

```
def add(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(A))] for i in range(len(A))]

def sub(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A))] for i in range(len(A))]

def strassen(A, B):
    if len(A) == 1:
        return [[A[0][0] * B[0][0]]]

    n = len(A)
    mid = n // 2

    A11 = [row[:mid] for row in A[:mid]]
    A12 = [row[mid:] for row in A[:mid]]
    A21 = [row[:mid] for row in A[mid:]]
    A22 = [row[mid:] for row in A[mid:]]

    B11 = [row[:mid] for row in B[:mid]]
    B12 = [row[mid:] for row in B[:mid]]
    B21 = [row[:mid] for row in B[mid:]]
    B22 = [row[mid:] for row in B[mid:]]

    M1 = strassen(add(A11, A22), add(B11, B22))
    M2 = strassen(add(A21, A22), B11)
    M3 = strassen(A11, sub(B12, B22))
    M4 = strassen(A22, sub(B21, B11))
    M5 = strassen(add(A11, A12), B22)
    M6 = strassen(sub(A21, A11), add(B11, B12))
    M7 = strassen(sub(A12, A22), add(B21, B22))

    C11 = add(sub(add(M1, M4), M5), M7)
    C12 = add(M3, M5)
    C21 = add(M2, M4)
    C22 = add(sub(add(M1, M3), M2), M6)
```

```
C = []
for i in range(mid):
    C.append(C11[i] + C12[i])
for i in range(mid):
    C.append(C21[i] + C22[i])

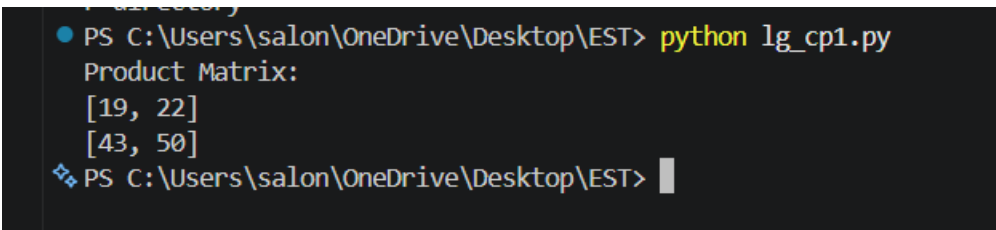
return C

A = [[1, 2],
      [3, 4]]

B = [[5, 6],
      [7, 8]]

result = strassen(A, B)

print("Product Matrix:")
for row in result:
    print(row)
```

Output:

```
PS C:\Users\salon\OneDrive\Desktop\EST> python lg_cp1.py
Product Matrix:
[19, 22]
[43, 50]
PS C:\Users\salon\OneDrive\Desktop\EST>
```

Q2. A company has only one conference room that can be used for meetings. Each meeting request has a start time and end time , and no two meetings can overlap. Given a list of meeting requests, maximize the number of non-overlapping mappings that can be scheduled in the room. You must choose the maximum number of meetings that can be accommodated without overlaps.

Code:

```
meetings = [(1, 2), (3, 4), (0, 6), (5, 7), (8, 9), (5, 9)]

meetings.sort(key=lambda x: x[1])

count = 1
selected = [meetings[0]]
last_end = meetings[0][1]

for start, end in meetings[1:]:
    if start >= last_end:
        selected.append((start, end))
```

```
count += 1
last_end = end
```

```
print("Maximum meetings:", count)
print("Selected meetings:")
for m in selected:
    print(m)
```

Output:

```
[43, 50]
PS C:\Users\salon\OneDrive\Desktop\EST> python lg_cp1.py
Maximum meetings: 4
Selected meetings:
(1, 2)
(3, 4)
(5, 7)
(8, 9)
PS C:\Users\salon\OneDrive\Desktop\EST>
```